

2026 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Утверждаю

Заведующий кафедрой

ИУ7

(индекс)

Рудаков И. В.

(И.О. Фамилия)

24.02.2026

(дата)

(подпись)

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Базы данных»

Студент группы ИУ7-62Б Диваев Александр Николаевич

(Фамилия, имя, отчество)

Тема курсовой работы

Разработка базы данных для анализа отказоустойчивости систем на основе микросервисной архитектуры

Направленность КР (учебная, исследовательская, практическая, производственная, др.)
учебная

Источник тематики (кафедра, кафедра
предприятие, НИР)

Задание

Сформулировать требования и ограничения к разрабатываемой базе данных и приложению для анализа отказоустойчивости систем на основе микросервисной архитектуры. Провести сравнительный анализ реляционных и графовых моделей данных для задач хранения сетевой топологии. Спроектировать сущности базы данных и ограничения целостности топологии сети. Спроектировать ролевую модель. Выбрать средства реализации базы данных и приложения. Описать методы тестирования разработанного функционала и разработать тесты для проверки корректности работы приложения. Исследовать зависимость времени выполнения запросов поиска зависимостей от количества узлов системы.

Оформление курсовой работы:

Расчетно-пояснительная записка на 18-40 листах формата А4. Презентация к курсовой работе на 6-15 слайдах.

Дата выдачи задания « 24 » февраля 2026 г.

Руководитель курсовой работы

(подпись, дата)

Мальцева Д.Ю.

(И.О. Фамилия)

Студент

(подпись, дата)

Диваев А.Н.

(И.О. Фамилия)

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсовой работы**

по дисциплине «Базы данных»
Студент группы ИУ7-62Б Диваев Александр Николаевич
(Фамилия, имя, отчество)

Тема курсовой работы
Разработка базы данных для анализа отказоустойчивости систем на основе микросервисной архитектуры

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель КР	Куратор
1.	Задание на выполнение курсовой работы	24.02.2026		Мальцева Д.Ю.	
2.	1 модуль	30.03.2026 <i>Планируемая дата</i>		Мальцева Д.Ю.	
3.	2 модуль	27.04.2026 <i>Планируемая дата</i>		Мальцева Д.Ю.	
4.	Оформление РПЗ	04.05.2026 <i>Планируемая дата</i>		Мальцева Д.Ю.	
5.	Подготовка доклада и презентации (при необходимости)	11.05.2026 <i>Планируемая дата</i>		Мальцева Д.Ю.	
6.	Защита курсовой работы	25.05.2026 <i>Планируемая дата</i>		Мальцева Д.Ю.	

Студент _____
(подпись, дата)

Руководитель работы _____
(подпись, дата)

РЕФЕРАТ

Расчётно-пояснительная записка 64 страницы, 16 рисунков, 17 таблиц, 21 источник, 3 приложения.

МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ОТКАЗОУСТОЙЧИВОСТЬ, ГРАФОВАЯ СУБД, РЕЛЯЦИОННАЯ СУБД, АЛГОРИТМЫ ОБХОДА ГРАФА, ПРОАКТИВНЫЙ АНАЛИЗ, NEO4J, POSTGRESQL, C#, .NET

Объектом исследования является топология ИТ-инфраструктуры распределенных информационных систем на основе микросервисной архитектуры. Объектом разработки является база данных и прикладное программное обеспечение для анализа отказоустойчивости и моделирования каскадных сбоев в распределенных системах.

Цель работы — проектирование и разработка базы данных, поддерживающей хранение высокосвязных данных и обеспечивающей высокую скорость выполнения алгоритмов обхода графа для поиска критических уязвимостей.

В аналитическом разделе проведен анализ предметной области управления надежностью систем и существующих коммерческих решений, показавший необходимость разработки проактивного инструмента для анализа топологии на этапе проектирования. Сформулированы требования к данным и ролевой модели пользователей. На основе анализа выбрана концепция полиглотного хранения данных, сочетающая реляционную и графовую модели.

В конструкторском разделе спроектирована структура хранения реляционной и графовой баз данных. На формальном языке описаны ограничения целостности и ролевая модель разграничения прав доступа на уровне приложения. Проведена проверка нахождения спроектированной реляционной базы данных в третьей нормальной форме.

В технологическом разделе обоснован выбор СУБД PostgreSQL и Neo4j, а также языковых средств разработки: платформы .NET 10, языка C# и объектно-реляционного отображения Entity Framework Core. Представлены исходные коды описания схем данных, а также тексты спроектированных запросов с использованием библиотеки процедур АРОС. Описана методика тестирования и приведены примеры работы интерфейса приложе-

ния.

В исследовательском разделе проведена практическая оценка производительности графовых запросов. Исследована зависимость времени выполнения анализа каскадных отказов от общего числа узлов системы, а также зависимость времени поиска циклических зависимостей от количества циклов в графе. Определено, что время выполнения сложных аналитических расчетов на топологиях объемом до десяти тысяч узлов не превышает нескольких десятков миллисекунд.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	9
ВВЕДЕНИЕ	10
1 Аналитический раздел	11
1.1 Анализ предметной области	11
1.2 Анализ существующих решений	11
1.3 Ролевая модель	12
1.4 Формализация данных	13
1.5 Анализ моделей баз данных	15
1.5.1 Дореляционная модель	15
1.5.2 Реляционные модели	15
1.5.3 Постреляционная модель	15
1.6 Требования и ограничения к разрабатываемой базе данных .	16
1.7 Формализация задачи	16
Выводы по аналитическому разделу	17
2 Конструкторский раздел	18
2.1 Проектирование базы данных	18
2.2 Ограничения целостности данных	22
2.2.1 Целостность таблиц и узлов	22
2.2.2 Целостность полей	22
2.2.3 Целостность ссылок	22
2.3 Ролевая модель на уровне приложения	23
2.4 Проверка нахождения реляционной БД в ЗНФ	25
2.4.1 Отношение «Команды»	26
2.4.2 Отношение «Пользователи»	27
2.4.3 Отношение «Приглашения»	27
2.4.4 Отношение «Системы»	28
2.4.5 Выводы по результатам анализа	29
Выводы по конструкторскому разделу	29

3	Технологический раздел	30
3.1	Выбор СУБД	30
3.1.1	Выбор реляционной СУБД	30
3.1.2	Выбор графовой СУБД	31
3.2	Выбор средств реализации приложения	32
3.3	Создание объектов БД	33
3.3.1	Создание объектов реляционной БД	33
3.4	Разработка запросов к графовой БД	35
3.5	Реализация ролевой модели и управления доступом	37
3.5.1	Реализация сущностей управления доступом	37
3.5.2	Логика работы системы приглашений	38
3.5.3	Разграничение прав доступа при выполнении операций	39
3.6	Тестирование программного обеспечения	40
3.6.1	Модульное тестирование	40
3.6.2	Интеграционное тестирование	41
3.7	Демонстрация работы приложения	41
	Выводы по технологическому разделу	46
4	Исследовательский раздел	47
4.1	Технические характеристики	47
4.2	Технология замеров времени	47
4.3	Первое исследование	47
4.3.1	Описание исследования	47
4.3.2	Результаты исследования	48
4.3.3	Вывод	49
4.4	Второе исследование	49
4.4.1	Описание исследования	49
4.4.2	Результаты исследования	50
4.4.3	Вывод	51
	Выводы по исследовательскому разделу	51
	ЗАКЛЮЧЕНИЕ	53
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	56
	ПРИЛОЖЕНИЕ А	57

ПРИЛОЖЕНИЕ Б	59
ПРИЛОЖЕНИЕ В	64

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения.

- API — Application Programming Interface — программный интерфейс приложения
- БД — База Данных
- СУБД — Система Управления Базами Данных
- ИТ — Информационные Технологии
- SRE — Site Reliability Engineering — дисциплина, разработанная для обеспечения надежности высоконагруженных информационных систем
- UUID — Universally Unique Identifier — универсальный уникальный идентификатор

ВВЕДЕНИЕ

Современные системы нередко строятся на основе микросервисной [1] архитектуры, состоящей из различных, связанных друг с другом, компонентов, например микросервисов, баз данных, внешних API, брокеров сообщений и других. В масштабных системах количество связей между компонентами может быть огромным.

Одним из ключевых требований к любой программной системе является надежность и отказоустойчивость. При большом количестве связей ручной анализ зависимостей компонентов друг от друга становится достаточно кропотливой задачей. Решением данной проблемы является автоматизация поиска зависимостей конкретных компонентов и симуляция каскадных сбоев, что сведет к минимуму вероятность ошибок вследствие человеческого фактора и уменьшит временные затраты на проектирование и анализ архитектуры системы.

Цель курсовой работы — разработать базу данных для анализа отказоустойчивости систем на основе микросервисной архитектуры. Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области;
- провести анализ существующих решений;
- сформулировать требования и ограничения к разрабатываемой базе данных;
- спроектировать ролевую модель на уровне приложения;
- выбрать СУБД для реализации разрабатываемой базы данных;
- спроектировать сущности базы данных и ограничения целостности топологии сети;
- сформулировать требования и ограничения к разрабатываемому приложению;
- выбрать средства реализации базы данных и приложения;
- описать методы тестирования разработанной функциональности и разработать тесты для проверки корректности работы приложения;
- исследовать зависимость времени выполнения запросов поиска зависимостей от количества узлов системы.

1 Аналитический раздел

1.1 Анализ предметной области

Предметная область проекта охватывает управление надежностью распределенных программных систем и анализ топологии микросервисной архитектуры. Современные системы нередко строятся на основе множества взаимодействующих компонентов, что значительно усложняет процесс контроля их работоспособности. Система предназначена для моделирования ИТ-инфраструктуры, что позволяет автоматизировать поиск единых точек отказа и прогнозировать распространение каскадных сбоев.

Основными сущностями предметной области являются компоненты и связи между ними. Компонент может представлять собой микросервис, базу данных, брокер сообщений или внешний программный интерфейс. Связь между компонентами показывает зависимость одного элемента от другого и может иметь различный уровень критичности.

1.2 Анализ существующих решений

В сфере обеспечения отказоустойчивости распределенных систем существует некоторое количество решений, являющихся стандартом в данной области, например ITRS Geneos [2] и связка Prometheus [3] с Grafana [4].

Для изучения актуальности работы было проведено сравнение существующих решений по следующим критериям: основное назначение, модель данных, метод выявления проблем, анализ влияния сбоев. В результате проведенного сравнения была составлена таблица 1.

Таблица 1 – Сравнение существующих решений

Критерий сравнения	ITRS Geneos	Prometheus + Grafana
Основное назначение	Контроль состояния серверов и приложений в реальном времени.	Сбор и визуализация метрик, анализ трендов.
Модель данных	Иерархическая	Временные ряды

Продолжение таблицы 1

Критерий сравнения	ITRS Geneos	Prometheus + Grafana
Метод выявления проблем	Реактивный. Сравнение текущих значений с пороговыми и правилами.	Реактивный. Мониторинг, отклонений, оповещения.
Анализ влияния	Ручной / Правила. Требуется сложной настройки корреляции событий.	Отсутствует. Показывает состояние метрик, но не моделирует зависимости сбоев.

В результате проведения анализа было выявлено, что существующие решения позволяют проводить только реактивный анализ и не дают возможности изучать узкие места системы на этапе проектирования. В связи с этим было решено, что разрабатываемая система должна предоставлять возможность проактивного анализа для выявления уязвимостей и архитектурных проблем до развертывания инфраструктуры.

1.3 Ролевая модель

Пользователи проектируемого приложения разделяются на несколько ролей в зависимости от их прав доступа:

- незарегистрированный пользователь — имеет возможность только пройти процесс авторизации в системе или зарегистрироваться по коду приглашения;
- разработчик — имеет доступ к просмотру топологии только тех систем, за которые отвечает его команда, и использует приложение для понимания последствий обновления своих сервисов;
- SRE-инженер — обладает правами разработчика, а также имеет возможность запускать алгоритмы анализа отказоустойчивости, моделировать сбои и просматривать формируемые отчеты;
- архитектор — обладает правами SRE-инженера, дополнительно имеет полные права на редактирование графа инфраструктуры, включая создание, изменение и удаление компонентов и связей;

- администратор — управляет глобальными настройками, учетными записями пользователей, создает команды, генерирует приглашения и назначает роли.

На рисунке А.2 представлена диаграмма вариантов использования для описанных ролей.

1.4 Формализация данных

Для реализации заявленной функциональности по разграничению доступа и моделированию инфраструктуры необходимо определить структуру сохраняемой информации. Выделим сущности для управления доступом: пользователь, команда, приглашение и система. Их описание приведено в таблице 2.

Таблица 2 – Сущности системы для управления доступом

Сущность	Информация
Пользователь	Идентификатор, имя пользователя, электронная почта, хэш пароля, идентификатор роли, идентификатор команды
Команда	Идентификатор, название команды, описание
Приглашение	Идентификатор, код приглашения, электронная почта получателя, роль нового пользователя, срок действия, статус активности, идентификатор целевой команды, идентификатор активировавшего пользователя
Система	Идентификатор, название, описание, дата создания, дата изменения, идентификатор команды-владельца

На рисунке А.1 приведена диаграмма сущность-связь для выделенных сущностей управления доступом.

Для моделирования ИТ-инфраструктуры выделены сущности, представляющие собой элементы графа: компонент и связь. Их описание приведено в таблице 3.

Таблица 3 – Сущности топологии инфраструктуры

Сущность	Информация
Компонент	Идентификатор, название, тип компонента, краткое описание, идентификатор родительской системы
Связь	Идентификатор, идентификатор исходного компонента, идентификатор целевого компонента, тип протокола, уровень критичности связи

На рисунке 1 представлена диаграмма сущность-связь графовой модели данных.

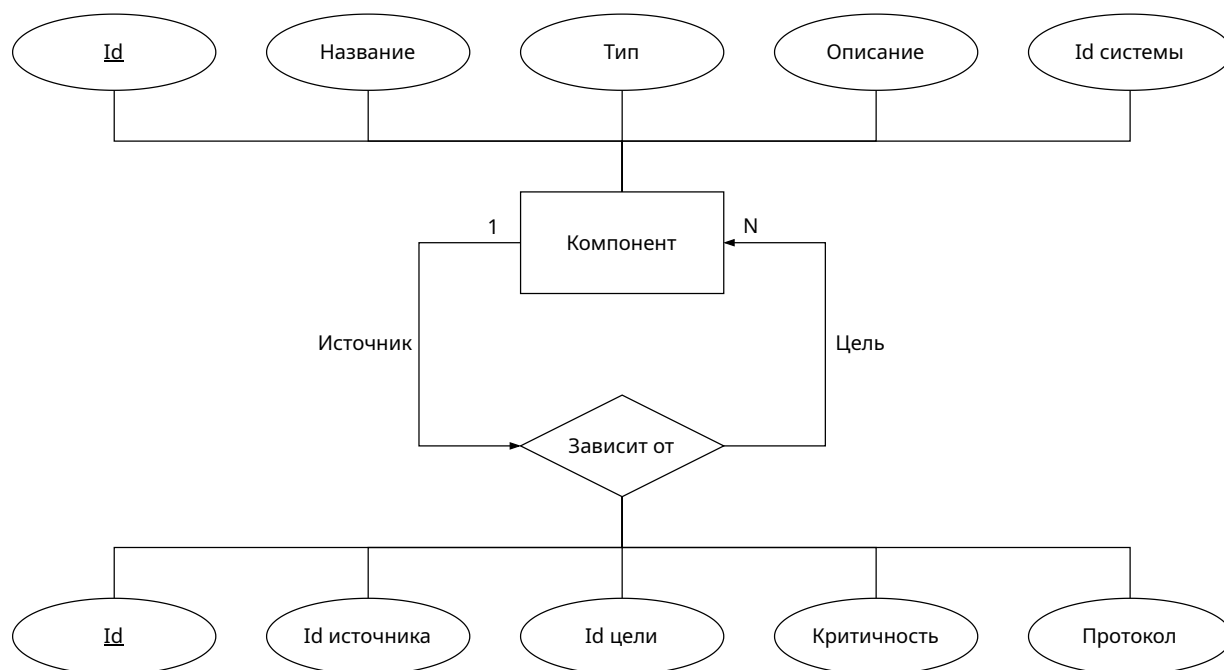


Рисунок 1 – Диаграмма сущность-связь графовой модели данных

1.5 Анализ моделей баз данных

База данных представляет собой самодокументированное собрание интегрированных записей, структурированное согласно определенной модели. Традиционно выделяют три этапа развития моделей данных: дореляционный, реляционный и постреляционный [5].

1.5.1 Дореляционная модель

К дореляционным моделям относятся иерархическая и сетевая. Иерархическая модель строится по принципу древовидной структуры, где записи связаны отношениями предок-потомок. Реализуется связь типа один-ко-многим, при этом каждый потомок имеет строго одного предка. Сетевая модель расширяет иерархическую, позволяя реализовывать связи типа многие-ко-многим. В такой модели запись может иметь несколько предков, что превращает структуру в граф. Достоинством дореляционных моделей является высокая скорость доступа к данным, а к недостаткам относятся значительные затраты памяти на хранение указателей и жесткость схемы.

1.5.2 Реляционные модели

В реляционной модели данные представляются в виде совокупности взаимосвязанных таблиц, а манипулирование ими осуществляется с помощью языка структурированных запросов. Основные понятия модели включают домен, атрибут, кортеж, первичный и внешний ключи. Преимуществами реляционной модели являются высокая согласованность данных, простота инструментария и независимость прикладного программного обеспечения от физического способа хранения данных. Недостатками являются снижение производительности при росте объема таблиц и жесткие ограничения на структуру.

1.5.3 Постреляционная модель

Постреляционные модели призваны преодолеть ограничения реляционного подхода, обеспечивая гибкость при работе со сложными структурами. В рамках постреляционного подхода выделяют графовую модель, где данные

представляются в виде узлов и ребер, обладающих собственными свойствами. Такая система оптимальна для работы с высокосвязными данными, где важна скорость обхода сложных структур. Достоинствами постреляционных моделей являются эффективная обработка информации со сложными связями, а основным недостатком является сложность обеспечения строгой целостности данных по сравнению с реляционными моделями.

1.6 Требования и ограничения к разрабатываемой базе данных

На основе структуры выделенных данных (компонентов и связей), представляющих собой ориентированную сеть, основным требованием к разрабатываемой базе данных является возможность оптимизированного хранения графов и эффективного выполнения глубокого обхода по ним. Это делает классические реляционные базы данных неэффективными из-за медленных рекурсивных запросов.

В то же время, для обеспечения безопасности и разграничения доступа к данным требуется спроектировать структурированное и непротиворечивое хранилище для пользователей, команд и приглашений.

На основе поставленных требований было принято решение использовать две СУБД для хранения данных. Для хранения топологии инфраструктуры и выполнения алгоритмического анализа будет использоваться графовая система управления базами данных, а для хранения учетных записей пользователей и ролей будет применена реляционная система управления базами данных.

1.7 Формализация задачи

На основе проведенного анализа предметной области, существующих решений и требований к базам данных сформулируем постановку задачи на разработку. В рамках курсовой работы необходимо спроектировать базы данных и разработать приложение для анализа отказоустойчивости на основе графов зависимостей.

В приложении должна быть реализована возможность регистрации и авторизации. Для изоляции данных друг от друга в системе должна быть

реализована концепция команд. Пользователи объединяются в команды, и каждая команда имеет доступ только к тем архитектурным схемам систем, владельцем которых она является.

Для добавления новых пользователей в команды, для администраторов должна быть реализована возможность генерации уникальных приглашений. Использование кода приглашения позволяет новому сотруднику зарегистрироваться и автоматически получить необходимую роль и привязку к команде.

Основная функциональность приложения должна включать создание систем, добавление в них компонентов и настройку связей между ними. В приложении должна быть реализована возможность запуска анализа последствий отказа выбранного компонента, выявления единых точек отказа и обнаружения недопустимых циклических зависимостей для выбранной топологии системы.

Выводы по аналитическому разделу

В аналитическом разделе был проведен анализ предметной области управления надежностью распределенных систем и рассмотрены существующие решения. Сформулированы требования к проектируемому приложению, разработана ролевая модель пользователей и формализованы данные, подлежащие хранению. Был проведен обзор дореляционных, реляционных и постреляционных моделей баз данных. На основе специфики задачи было принято решение о разделении данных: применении графовой модели для хранения архитектурных зависимостей и реляционной модели для управления метаданными и правами доступа пользователей.

2 Конструкторский раздел

В данном разделе приводится описание структуры разрабатываемой базы данных. В соответствии с аналитическим разделом была выбрана гибридная модель хранения данных. Для реляционной базы данных каждой выделенной сущности ставится в соответствие таблица, а для графовой базы данных описываются структуры узлов и ребер. Рассматриваются спроектированные ограничения целостности, а также ролевая модель управления доступом на уровне приложения.

2.1 Проектирование базы данных

На основе формализации данных, проведенной в аналитическом разделе, в разрабатываемой реляционной базе данных выделены следующие таблицы:

- teams — таблица команд разработчиков;
- users — таблица пользователей приложения;
- invites — таблица приглашений для регистрации;
- it_systems — таблица систем.

В таблицах 4 — 7 определены типы данных и ограничения для каждого столбца перечисленных реляционных таблиц.

Таблица 4 – Информация о таблице teams

Столбец	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, первичный ключ
name	Название	Строка	Не null, уникальный
description	Описание	Строка	

Таблица 5 – Информация о таблице users

Столбец	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, первичный ключ
username	Имя пользователя	Строка	Не null, уникальный
email	Электронная почта	Строка	Не null, уникальный
password_hash	Хэш пароля	Строка	Не null
role	Идентификатор роли	Целое число	Не null
team_id	Идентификатор команды	UUID	Внешний ключ

Таблица 6 – Информация о таблице invites

Столбец	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, первичный ключ
status	Статус приглашения	Целое число	Не null
target_email	Электронная почта получателя	Строка	Не null, уникальный
role	Роль нового пользователя	Целое число	Не null
code	Уникальный код	Строка	Не null, уникальный
expiration_date	Дата истечения	Дата и время	Не null
team_id	Идентификатор команды	UUID	Не null, внешний ключ
activated_by_uid	Идентификатор нового пользователя	UUID	Не null, внешний ключ

Таблица 7 – Информация о таблице it_systems

Столбец	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, первичный ключ
name	Название	Строка	Не null
description	Описание	Строка	
created_at	Дата создания	Дата и время	Не null
updated_at	Дата обновления	Дата и время	Не null
team_id	Идентификатор команды	UUID	Не null, внешний ключ

Для хранения топологии инфраструктуры в графовой базе данных спроектирована структура узлов и связей. Данные сущности не имеют жесткой табличной схемы, но подчиняются правилам модели графа свойств.

В графовой базе данных определен узел Component, атрибуты которого представлены в таблице 8.

Таблица 8 – Информация об атрибутах узла Component

Атрибут	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, уникальный
name	Название	Строка	Не null
type	Тип компонента	Число	Не null
description	Описание	Строка	
system_id	Идентификатор системы	UUID	Не null

Связь между узлами определяется направленным ребром DEPENDS_ON. Информация об атрибутах данного ребра представлена в таблице 9.

Таблица 9 – Информация об атрибутах связи DEPENDS_ON

Атрибут	Значение	Тип	Ограничение
id	Идентификатор	UUID	Не null, уникальный
protocol	Тип протокола	Число	Не null
severity	Уровень критичности	Число	Не null

На рисунке 2 представлена полная схема разрабатываемой реляционной базы данных.

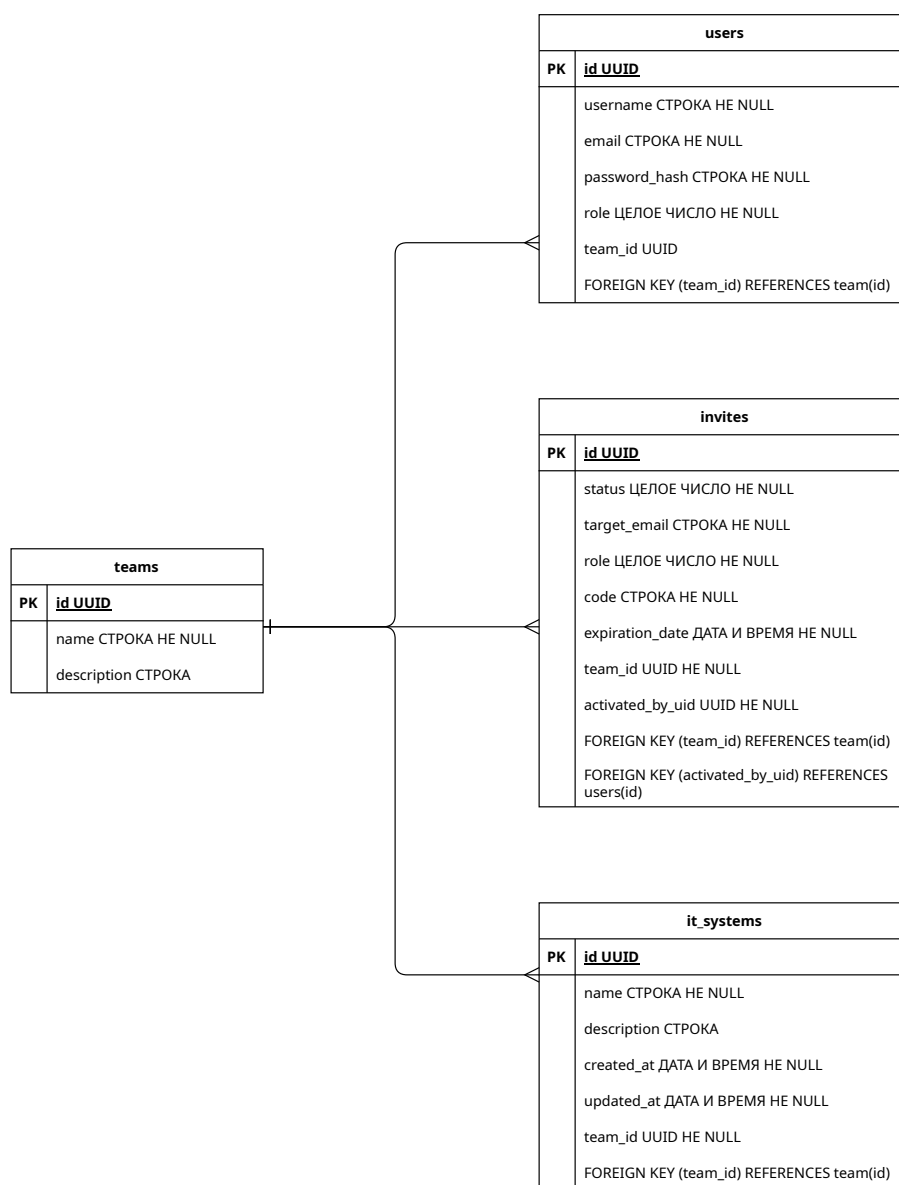


Рисунок 2 – Схема реляционной БД в нотации DBML

2.2 Ограничения целостности данных

2.2.1 Целостность таблиц и узлов

Для обеспечения целостности реляционных таблиц каждая строка имеет уникальный идентификатор, являющийся первичным ключом. Первичные ключи назначены полям `id` во всех спроектированных таблицах. Для графовой базы данных на уровне СУБД устанавливается ограничение уникальности на свойство `id` для всех узлов с меткой `Component`, что предотвращает дублирование узлов графа.

2.2.2 Целостность полей

Для корректного функционирования системы накладываются ограничения на значения отдельных полей. В таблице `users` поля `username` и `email` должны быть уникальными для предотвращения создания дублирующихся учетных записей. В таблице `invites` поля `code` и `target_email` также являются уникальным, а значение поля `expiration_date` должно быть строго больше даты создания приглашения. В таблице `it_systems` дата создания не может превышать текущее системное время. В графовой базе данных атрибут `type` узла `Component` может принимать только определенные значения: микросервис, база данных, брокер сообщений, внешний API.

2.2.3 Целостность ссылок

Обеспечение ссылочной целостности в реляционной базе данных реализуется с помощью внешних ключей. Для каждого значения внешнего ключа в дочерней таблице должна существовать запись с соответствующим первичным ключом в родительской таблице. В таблице 10 представлены все внешние ключи реляционной схемы.

Таблица 10 – Внешние ключи таблиц реляционной базы данных

Таблица	Внешний ключ	Ссылка на родительскую таблицу
users	teams_id	teams (id)
invites	teams_id	teams (id)
invites	activated_by_uid	users (id)
it_systems	teams_id	teams (id)

2.3 Ролевая модель на уровне приложения

Для обеспечения безопасности и разграничения доступа к информации реализована ролевая модель управления доступом на программном уровне. Базы данных не содержат индивидуальных учетных записей для каждого пользователя системы. Вместо этого приложение подключается к хранилищам данных с использованием единых учетных записей, а вся логика проверки прав доступа и фильтрации запросов выполняется на уровне контроллеров и сервисов разрабатываемого приложения.

В системе определены уровни доступа для пользователей, описанные в таблице 11.

Таблица 11 – Описание ролей системы

Роль	Права
Незарегистрированный пользователь	Имеет доступ исключительно к функциям регистрации, авторизации и валидации кода приглашения.

Продолжение таблицы 11

Роль	Права
Разработчик	Обладает правами на чтение данных о пользователях своей команды, а также на просмотр архитектурных схем систем, привязанных к его команде. Пользователь с данной ролью может инициировать запросы на визуализацию топологии, но не имеет прав на внесение изменений в структуру графа.
SRE	Наследует все права разработчика, а также получает доступ к аналитическим функциям приложения. Данная роль позволяет запускать алгоритмические расчеты каскадных отказов, моделировать сценарии сбоя компонентов и запрашивать вычисление критических путей и единых точек отказа в графе.
Архитектор	Обладает расширенными правами, включающими в себя возможности SRE-инженера, а также права на изменение топологии. Архитектор может отправлять запросы на создание, обновление и удаление систем, добавление новых узлов и настройку связей между ними в графовой базе данных.

Продолжение таблицы 11

Роль	Права
Администратор	Обладает полными правами в системе управления доступом. Пользователь с этой ролью может просматривать список всех зарегистрированных пользователей, создавать и удалять команды, генерировать коды приглашений.

2.4 Проверка нахождения реляционной БД в ЗНФ

Для каждого отношения $R(U, F)$ разработанной реляционной базы данных проверка условий ЗНФ выполняется следующим образом:

Классификация атрибутов отношения

Все атрибуты из U на основе их участия в функциональных зависимостях из множества F делятся на три непересекающиеся группы:

- G_1 : атрибуты, встречающиеся только в левых частях ФЗ, либо вообще не вошедшие в функциональные зависимости из множества F , эти атрибуты входят в состав всех потенциальных ключей отношения;
- G_2 : атрибуты, встречающиеся только в правых частях ФЗ, данные атрибуты не входят в состав потенциальных ключей;
- G_3 : промежуточные атрибуты, которые встречаются как в левых, так и в правых частях ФЗ из множества F .

Определение потенциальных ключей

Вычисляется замыкание множества атрибутов первой группы: $(G_1)^+$.

- если $(G_1)^+ = U$, то множество G_1 является единственным минимальным потенциальным ключом отношения;

- если $(G_1)^+ \neq U$, то для получения потенциальных ключей базовая группа G_1 последовательно комбинируется с подмножествами атрибутов из промежуточной группы G_3 до тех пор, пока замыкание очередной комбинации не покроет все множество U .

Проверка требований ЗНФ

Отношение находится в ЗНФ тогда и только тогда, когда для любой нетривиальной функциональной зависимости $X \rightarrow Y \in F^+$ выполняется хотя бы одно из следующих условий:

- левая часть зависимости X является суперключом отношения R ;
- правая часть зависимости Y состоит исключительно из первичных атрибутов, входящих в состав хотя бы одного потенциального ключа.

2.4.1 Отношение «Команды»

Атрибуты:

$$U = \{\text{id}, \text{name}, \text{description}\}$$

Множество ФЗ:

$$F = \{\text{id} \rightarrow \{\text{name}, \text{description}\}\}$$

Классификация атрибутов:

- только в левой части: id;
- только в правой части: name, description;
- в обеих частях: $\emptyset$.

Нахождение потенциальных ключей:

$$(\text{id})^+ = \{\text{id}, \text{name}, \text{description}\} = U$$

Потенциальные ключи: id.

Проверка требований ЗНФ: левые части всех функциональных зависимостей являются потенциальными ключами, следовательно, суперключами отношения «Команды». Условие ЗНФ выполняется.

2.4.2 Отношение «Пользователи»

Атрибуты:

$$U = \{\text{id}, \text{username}, \text{email}, \text{password_hash}, \text{role}, \text{team_id}\}$$

Множество ФЗ:

$$F = \{\text{id} \rightarrow \{\text{username}, \text{email}, \text{password_hash}, \text{role}, \text{team_id}\},$$

$$\text{username} \rightarrow \{\text{id}, \text{email}, \text{password_hash}, \text{role}, \text{team_id}\},$$

$$\text{email} \rightarrow \{\text{id}, \text{username}, \text{password_hash}, \text{role}, \text{team_id}\}\}$$

Классификация атрибутов:

- только в левой части: $\emptyset$;
- только в правой части: $\text{password_hash}, \text{role}, \text{team_id}$;
- в обеих частях: $\text{id}, \text{username}, \text{email}$.

Нахождение потенциальных ключей:

$$(\text{id})^+ = U$$

$$(\text{username})^+ = U$$

$$(\text{email})^+ = U$$

Потенциальные ключи: $\text{id}, \text{username}$ и email .

Проверка требований ЗНФ: левые части всех функциональных зависимостей являются потенциальными ключами, следовательно, суперключами отношения «Пользователи». Условие ЗНФ выполняется.

2.4.3 Отношение «Приглашения»

Атрибуты:

$$U = \{\text{id}, \text{status}, \text{target_email}, \text{role}, \text{code}, \\ \text{expiration_date}, \text{team_id}, \text{activated_by_uid}\}$$

Множество ФЗ:

$$\begin{aligned} F = \{ & \text{id} \rightarrow \{\text{status}, \text{target_email}, \text{role}, \text{code}, \\ & \text{expiration_date}, \text{team_id}, \text{activated_by_uid}\}, \\ & \text{code} \rightarrow \{\text{id}, \text{status}, \text{target_email}, \text{role}, \\ & \text{expiration_date}, \text{team_id}, \text{activated_by_uid}\}, \\ & \text{target_email} \rightarrow \{\text{id}, \text{status}, \text{role}, \text{code}, \\ & \text{expiration_date}, \text{team_id}, \text{activated_by_uid}\} \} \end{aligned}$$

Классификация атрибутов:

- только в левой части: $\emptyset$;
- только в правой части: status, role, expiration_date, team_id, activated_by_uid;
- в обеих частях: id, code, target_email.

Нахождение потенциальных ключей:

$$(\text{id})^+ = U$$

$$(\text{code})^+ = U$$

$$(\text{target_email})^+ = U$$

Потенциальные ключи: id, code и target_email.

Проверка требований ЗНФ: левые части всех функциональных зависимостей являются потенциальными ключами отношения «Приглашения».

Условие ЗНФ выполняется.

2.4.4 Отношение «Системы»

Атрибуты:

$$U = \{\text{id}, \text{name}, \text{description}, \text{created_at}, \text{updated_at}, \text{team_id}\}$$

Множество ФЗ:

$$F = \{\text{id} \rightarrow \{\text{name}, \text{description}, \text{created_at}, \text{updated_at}, \text{team_id}\}\}$$

Классификация атрибутов:

- только в левой части: id;
- только в правой части: name, description, created_at, updated_at, team_id;
- В обеих частях: $\emptyset$.

Нахождение потенциальных ключей:

$$(\text{id})^+ = \{\text{id}, \text{name}, \text{description}, \text{created_at}, \text{updated_at}, \text{team_id}\} = U$$

Потенциальный ключ: id.

Проверка требований 3НФ: левая часть единственной функциональной зависимости совпадает с потенциальным ключом отношения «Системы». Транзитивные зависимости отсутствуют, условие 3НФ выполняется.

2.4.5 Выводы по результатам анализа

На основе проведенного анализа функциональных зависимостей и поиска потенциальных ключей для всех отношений реляционной базы данных сделаны следующие выводы:

- для каждого из четырех отношений левая часть любой нетривиальной ФЗ является суперключом соответствующего отношения;
- в схеме базы данных отсутствуют транзитивные ФЗ первичных атрибутов от потенциальных ключей.

Таким образом, спроектированная схема реляционной базы данных полностью удовлетворяет требованиям третьей нормальной формы.

Выводы по конструкторскому разделу

В данном разделе была спроектирована структура хранения данных для системы анализа отказоустойчивости. Описаны таблицы реляционной базы данных и структуры графовой базы данных, определены связи между ними и ограничения целостности. Формализована ролевая модель управления доступом на уровне приложения, обеспечивающая безопасность взаимодействия с системой и разграничение функциональных возможностей пользователей.

3 Технологический раздел

В данном разделе рассматриваются различные реляционные и графовые СУБД, проводится их сравнение по определенным критериям. Также описываются выбранные средства реализации и тестирование, демонстрируются результаты тестирования и функциональность разработанного приложения.

3.1 Выбор СУБД

В соответствии с аналитическим разделом для хранения данных разрабатываемой системы необходимо использовать две различные системы управления базами данных: реляционную и графовую.

3.1.1 Выбор реляционной СУБД

Для хранения информации о пользователях, ролях, командах и конфигурациях систем необходима реляционная система управления базами данных. Наиболее распространенными решениями в данной области являются:

- PostgreSQL [6];
- MySQL [7];
- Microsoft SQL Server [8];
- Oracle Database [9].

В таблице 12 приводится сравнение рассматриваемых реляционных систем по выделенным критериям.

Таблица 12 – Сравнение реляционных систем управления базами данных

Решение	Открытый исходный код и бесплатное распространение.	Поддержка стандарта SQL.	Кроссплатформенность и поддержка запуска в изолированных контейнерах.
PostgreSQL	Да	Да	Да

Продолжение таблицы 12

Решение	Открытый исходный код и бесплатное распространение.	Поддержка стандарта SQL.	Кроссплатформенность и поддержка запуска в изолированных контейнерах.
MySQL	Да	Частично	Да
Microsoft SQL Server	Нет	Да	Да
Oracle Database	Нет	Да	Да

В качестве используемой реляционной системы управления базами данных был выбран PostgreSQL, так как он имеет полностью открытый код, распространяется бесплатно и полностью поддерживает стандарт SQL.

3.1.2 Выбор графовой СУБД

Для хранения топологии инфраструктуры и выполнения алгоритмического обхода графа необходима графовая система управления базами данных. Наиболее популярными решениями на рынке являются:

- Neo4j [10];
- ArangoDB [11];
- OrientDB [12].

В таблице 13 приводится сравнение рассматриваемых графовых систем по выделенным критериям.

Таблица 13 – Сравнение графовых систем управления базами данных

Решение	Нативная графовая архитектура хранения и обработки данных.	Наличие встроенной библиотеки графовых алгоритмов.	Наличие бесплатной версии.
Neo4j	Да	Да	Да

Продолжение таблицы 13

Решение	Нативная графовая архитектура хранения и обработки данных.	Наличие встроенной библиотеки графовых алгоритмов.	Наличие бесплатной версии.
ArangoDB	Нет	Нет	Да
OrientDB	Нет	Нет	Да

Решения ArangoDB и OrientDB являются мультимодельными базами данных, совмещающими в себе документо-ориентированный и графовый подходы, что приводит к снижению производительности при глубоком рекурсивном обходе связей. СУБД Neo4j является полностью нативной графовой базой данных и предоставляет доступ к специализированной библиотеке графовых алгоритмов, идеально подходящей для вычисления единых точек отказа и выявления циклических зависимостей. По результатам сравнения в качестве графовой системы управления базами данных была выбрана Neo4j.

3.2 Выбор средств реализации приложения

В качестве языка программирования для разработки серверной части приложения был выбран C# [13]. Данный язык обладает строгой статической типизацией, богатым набором встроенных структур данных и поддержкой парадигм объектно-ориентированного и асинхронного программирования. Разработка ведется на базе современной кроссплатформенной платформы .NET 10 [14]. Выбор данной платформы обусловлен ее высокой производительностью, встроенной поддержкой внедрения зависимостей и развитой экосистемой, что делает ее оптимальным решением для создания надежных и масштабируемых серверных приложений.

Для взаимодействия с реляционной базой данных PostgreSQL была выбрана технология объектно-реляционного отображения Entity Framework Core [15]. Использование EF Core позволяет абстрагироваться от написания низкоуровневых SQL-запросов и обеспечивает манипулирование данными

посредством строго типизированных LINQ-запросов [16], а также автоматизируя процесс управления схемой базы данных через механизм миграций. В свою очередь, для интеграции с графовой базой данных Neo4j применяется официальный драйвер Neo4j.Driver [17]. Такое разделение средств доступа к данным позволяет эффективно реализовать паттерн репозитория, инкапсулируя специфику работы с каждой конкретной СУБД.

3.3 Создание объектов БД

Для PostgreSQL процесс создания таблиц и настройки ограничений возложен на используемый фреймворк EF Core. В графовой СУБД Neo4j таблиц нет, а структура всегда одинакова: узлы и связи между ними, поэтому описание структуры заранее не производится.

3.3.1 Создание объектов реляционной БД

Фреймворк EF Core использует механизм миграций. Миграция БД — это процесс переноса данных, структур или кода из одной среды, системы или формата в другую. В разработке механизм миграций используется как «версионный контроль» структуры БД с помощью кода.

Структура БД в фреймворке EF Core описывается на выбранном языке программирования. Согласно описанию фреймворк формирует SQL запросы для создания описанных объектов, вспомогательных таблиц для отслеживания миграций, ограничений, индексов. Каждое изменение структуры БД оформляется в виде отдельной миграции. Исходный код описания структуры разрабатываемой БД представлен в листинге 1.

Листинг 1 – Описание структуры реляционной БД

```
1 public class AnalyzerDbContext(DbContextOptions<AnalyzerDbContext>  
    options) : DbContext(options)  
2 {  
3     public DbSet<User> Users => Set<User>();  
4     public DbSet<Team> Teams => Set<Team>();  
5     public DbSet<Invite> Invites => Set<Invite>();  
6     public DbSet<ITSystem> ITSystems => Set<ITSystem>();  
7  
8     protected override void OnModelCreating(ModelBuilder  
        modelBuilder)
```

Листинг 1 (продолжение)

```

9      {
10          base.OnModelCreating(modelBuilder);
11
12          modelBuilder.Entity<User>(builder =>
13          {
14              builder.HasKey(u => u.Id);
15              builder.HasIndex(u => u.Username).IsUnique();
16              builder.HasIndex(u => u.Email).IsUnique();
17
18              builder.Property(u => u.Role).HasConversion<int>();
19
20              builder.HasOne<Team>()
21                  .WithMany()
22                  .HasForeignKey(u => u.TeamId)
23                  .OnDelete(DeleteBehavior.Restrict);
24          });
25
26          modelBuilder.Entity<Team>(builder =>
27          {
28              builder.HasKey(t => t.Id);
29
30              builder.Ignore(t => t.MemberIds);
31              builder.Ignore("_memberIds");
32          });
33
34          modelBuilder.Entity<Invite>(builder =>
35          {
36              builder.HasKey(i => i.Id);
37              builder.HasIndex(i => i.Code).IsUnique();
38
39              builder.Property(i => i.Status).HasConversion<int>();
40              builder.Property(i => i.Role).HasConversion<int>();
41
42              builder.HasOne<Team>()
43                  .WithMany()
44                  .HasForeignKey(i => i.TeamId)
45                  .OnDelete(DeleteBehavior.Cascade);
46
47              builder.HasOne<User>()
48                  .WithMany()
49                  .HasForeignKey(i => i.ActivatedByUserId)
50                  .OnDelete(DeleteBehavior.SetNull);
51          });
52
53          modelBuilder.Entity<ITSystem>(builder =>
54          {

```

Листинг 1 (продолжение)

```
55         builder.HasKey(s => s.Id);
56         builder.HasIndex(s => new { s.TeamId, s.Name
57             }).IsUnique();
58
59         builder.HasOne<Team>()
60             .WithMany()
61             .HasForeignKey(s => s.TeamId)
62             .OnDelete(DeleteBehavior.Cascade);
63     });
64 }
```

3.4 Разработка запросов к графовой БД

Для реализации функциональности анализа отказоустойчивости ИТ-инфраструктуры в графовой базе данных применяются специализированные запросы с использованием библиотеки расширенных процедур АРОС [18]. Применение данной библиотеки позволяет делегировать выполнение ресурсоемких операций обхода графа, фильтрации и поиска путей на сторону оптимизированного ядра СУБД, что значительно повышает производительность аналитической системы по сравнению с использованием стандартных транзитивных запросов языка Cypher [19].

Алгоритмы выявления последствий каскадных отказов реализован на основе процедуры извлечения подграфа. Данная процедура выполняет глубокий обход связей в обратном направлении, исследуя только те ребра, которые указывают на целевой компонент. В результате формируется плоский список всех микросервисов, которые прямо или косвенно зависят от указанного узла и могут перестать функционировать в случае его сбоя. Текст запроса представлен в листинге 2.

Листинг 2 – Текст запроса поиска каскадных отказов

```
1 MATCH (failed {id: $FailedId})
2 CALL apoc.path.subgraphNodes(failed, {
3     relationshipFilter: '<DEPENDS_ON',
4     minLevel: 1
5 }) YIELD node AS affected
6 RETURN affected.id AS Id
```

Для поиска единых точек отказа применяется аналогичный подход с последующей агрегацией результатов. Система обходит граф для каждого компонента, вычисляя суммарное количество транзитивно зависящих от него узлов. Если полученное значение превышает параметрически заданный порог критичности, данный компонент заносится в результирующую выборку как архитектурно уязвимое звено инфраструктуры, отказ которого нанесет наибольший ущерб системе. Текст запроса представлен в листинге 3.

Листинг 3 – Текст запроса поиска единых точек отказа

```
1 MATCH (c {system_id: $SystemId})
2 CALL apoc.path.subgraphNodes(c, {
3     relationshipFilter: '<DEPENDS_ON',
4     minLevel: 1
5 }) YIELD node AS dependent
6 WITH c, count(dependent) AS ImpactCount
7 WHERE ImpactCount >= $Threshold
8 RETURN c.id AS Id, ImpactCount
```

Для обнаружения недопустимых циклических зависимостей используется процедура конфигурационного расширения пути. Запрос инициирует глубокий поиск, устанавливая исходный узел в качестве терминального условия завершения обхода. Чтобы исключить бесконечное заикливание, применяется контроль уникальности на уровне пройденных ребер. Для устранения дублирующихся контуров в результатах выдачи производится лексикографическая сортировка идентификаторов узлов, формирующих найденный цикл. Текст запроса представлен в листинге 4.

Листинг 4 – Текст запроса поиска циклических зависимостей

```
1 MATCH (c {system_id: $SystemId})
2 CALL apoc.path.expandConfig(c, {
3     relationshipFilter: 'DEPENDS_ON>',
4     terminatorNodes: [c],
5     minLevel: 1,
6     uniqueness: 'RELATIONSHIP_PATH'
7 }) YIELD path
8 WITH [node in nodes(path)[0..-1] | node.id] AS cycleMembers
9 WITH apoc.coll.sort(cycleMembers) AS sortedCycle
10 RETURN DISTINCT sortedCycle AS CycleIds
```

При оценке рисков развертывания новых версий микросервисов применяется алгоритм поиска всех маршрутов, ведущих к целевому компоненту. Процедура извлекает полные цепочки зависимостей, отслеживая транзитивные вызовы. Перед возвратом результата массив узлов программно инвертируется средствами графовой базы данных, что позволяет наглядно предоставить порядок передачи запросов от первоначальных точек входа до обновляемого сервиса. Текст запроса представлен в листинге 5.

Листинг 5 – Текст запроса оценки рисков развертывания

```
1 MATCH (target {id: $TargetId})
2 CALL apoc.path.expandConfig(target, {
3     relationshipFilter: '<DEPENDS_ON',
4     minLevel: 1,
5     uniqueness: 'NODE_PATH'
6 }) YIELD path
7 RETURN [node in reverse(nodes(path)) | node.id] AS PathIds
```

3.5 Реализация ролевой модели и управления доступом

В разрабатываемой системе архитектура управления доступом и ролевая модель реализованы на уровне приложения, что позволяет абстрагироваться от механизмов конкретных систем управления базами данных. В основе реализации лежит объектно-ориентированный подход и принципы предметно-ориентированного проектирования. Логика авторизации, изоляции данных и распределения прав инкапсулирована в доменных сущностях и специализированных сервисах приложения.

3.5.1 Реализация сущностей управления доступом

Для обеспечения изоляции данных и гибкого управления правами были разработаны классы доменных сущностей, отражающие концепцию командной работы.

Сущность пользователя приложения инкапсулирует базовые учетные данные, такие как имя пользователя, адрес электронной почты и хэш пароля. Важной архитектурной особенностью является то, что каждый рядовой

пользователь жестко привязан к конкретной команде разработчиков через соответствующий идентификатор, а также имеет назначенную роль: разработчик, инженер по надежности или архитектор. Исключением является роль администратора. Объект администратора создается без привязки к команде, поскольку пользователи с данной ролью имеют глобальный доступ к управлению всей системой и не участвуют в разработке конкретных проектов.

Сущность команды представляет собой агрегатор, объединяющий группу пользователей. Команда выступает в качестве владельца для инфраструктурных систем. Разработанная логика на уровне сервисов гарантирует, что команда не может быть удалена из базы данных, если за ней закреплена хотя бы одна спроектированная ИТ-система. Это защищает топологические данные от потери связи с владельцами.

Сущность системы описывает инфраструктурный проект и содержит уникальный идентификатор команды-владельца. При выполнении запросов на чтение или изменение архитектурного графа на уровне сервисов приложения всегда происходит проверка принадлежности запрашиваемой системы к команде текущего авторизованного пользователя.

3.5.2 Логика работы системы приглашений

С целью повышения уровня информационной безопасности свободная регистрация в приложении запрещена. Процесс добавления новых пользователей реализован через защищенный механизм приглашений, логика которого инкапсулирована в отдельной доменной сущности и соответствующем сервисе.

Сущность приглашения содержит целевой адрес электронной почты, уникальный код, срок действия, идентификатор целевой команды и назначаемую роль. Приглашение реализует конечный автомат состояний и может находиться в одном из четырех статусов: в ожидании, просрочено, активировано или отозвано. Переход между состояниями строго контролируется внутренней логикой класса.

Жизненный цикл регистрации нового сотрудника состоит из следующих этапов:

- администратор системы формирует запрос на создание пригла-

шения, указывая адрес электронной почты нового сотрудника, команду и необходимый уровень доступа;

- приложение генерирует уникальный код, фиксирует срок годности ссылки и сохраняет объект приглашения в базу данных;
- при попытке регистрации пользователь передает полученный код, свои личные данные и пароль;
- сервис валидации проверяет существование кода, соответствие введенного адреса электронной почты целевому адресу в приглашении, а также текущий статус приглашения и его срок годности;
- в случае успешной проверки система применяет алгоритм усиленного хэширования к паролю пользователя, создает новую учетную запись и автоматически добавляет пользователя в список участников целевой команды;
- статус приглашения необратимо переводится в активированное состояние с фиксацией идентификатора зарегистрированного пользователя, что исключает возможность повторного использования кода.

3.5.3 Разграничение прав доступа при выполнении операций

После успешной аутентификации пользователя приложение генерирует и возвращает клиенту подписанный токен доступа, содержащий идентификатор пользователя, его роль и идентификатор команды. В дальнейшем данный токен прикрепляется к каждому сетевому запросу.

Механизм распределения прав работает на уровне контроллеров и сервисов приложения. При попытке доступа к конечным точкам прикладного программного интерфейса система извлекает роль из токена и сопоставляет ее с требуемым уровнем доступа для запрашиваемой операции.

Пользователи с ролью разработчика имеют доступ только к операциям чтения данных о компонентах и связях систем своей команды. Инженеры по надежности получают дополнительный доступ к конечным точкам аналитических сервисов, которые иницируют выполнение процедур обхода графа в базе данных для поиска циклических зависимостей и единых точек отказа. Архитекторы обладают правами на вызов операций модификации графа,

таких как добавление новых узлов и настройка протоколов взаимодействия между ними. Администраторы изолированы от функций работы с архитектурными графами, но имеют эксклюзивный доступ к сервисным функциям управления пользователями, отзыва приглашений и модификации состава команд.

Такой подход гарантирует соблюдение принципа наименьших привилегий и обеспечивает надежную изоляцию данных многопользовательской системы.

3.6 Тестирование программного обеспечения

Для обеспечения корректности работы системы было проведено тестирование разработанной функциональности. В качестве фреймворка для организации и выполнения тестов выбран xUnit [20]. Тестирование написанного кода разделено на два этапа: модульное для проверки изолированной бизнес-логики и интеграционное для проверки взаимодействия приложения с хранилищами данных.

3.6.1 Модульное тестирование

Модульное тестирование применяется для проверки классов сервисов, располагающихся на уровне логики приложения. Основной задачей данного вида тестирования является проверка правильности выполнения бизнес-правил, алгоритмов генерации ошибок и обработки исключительных ситуаций без обращения к реальной базе данных или внешним программным интерфейсам.

Для изоляции тестируемых компонентов применена библиотека Moq. Данный инструмент позволяет создавать фиктивные объекты зависимостей, подменяя реальные реализации интерфейсов репозиториев и вспомогательных провайдеров.

Тесты покрывают как позитивные сценарии, когда ожидается успешное выполнение операции и вызов соответствующих методов сохранения данных, так и негативные сценарии, проверяющие генерацию требуемых системных исключений при передаче некорректных входных параметров. Результаты модульного тестирования показаны на рисунке 3.

```
Analyzer.Tests net10.0 succeeded (0.7s) → bin/Debug/net10.0/Analyzer.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.4)
[xUnit.net 00:00:00.12]   Discovering: Analyzer.Tests
[xUnit.net 00:00:00.20]   Discovered:   Analyzer.Tests
[xUnit.net 00:00:00.25]   Starting:    Analyzer.Tests
[xUnit.net 00:00:02.96]   Finished:   Analyzer.Tests
Analyzer.Tests test net10.0 succeeded (3.9s)

Test summary: total: 88, failed: 0, succeeded: 88, skipped: 0, duration: 3.9s
```

Рисунок 3 – Результаты модульного тестирования

3.6.2 Интеграционное тестирование

Интеграционное тестирование направлено на проверку слоя инфраструктуры и доступа к данным, в частности, классов репозитория, взаимодействующих с базой данных посредством технологии объектно-реляционного отображения.

Для обеспечения консистентности проверок применяется механизм общих тестовых фикстур. Этот подход позволяет разворачивать изолированный контекст базы данных для набора тестов, гарантируя, что изменения данных в одном тесте не повлияют на результаты выполнения других. В ходе интеграционного тестирования проверяется корректность трансляции запросов из кода приложения в язык базы данных, а также правильность сохранения, обновления, поиска и удаления сущностей БД.

Результаты интеграционного тестирования показаны на рисунке 4.

```
[xUnit.net 00:00:19.38]   Finished:    Analyzer.IntegrationTests
Analyzer.IntegrationTests test net10.0 succeeded (20.3s)

Test summary: total: 23, failed: 0, succeeded: 23, skipped: 0, duration: 20.3s
Build succeeded in 23.7s
```

Рисунок 4 – Результаты интеграционного тестирования

3.7 Демонстрация работы приложения

На рисунках 5 – 12 представлен интерфейс реализованного приложения.

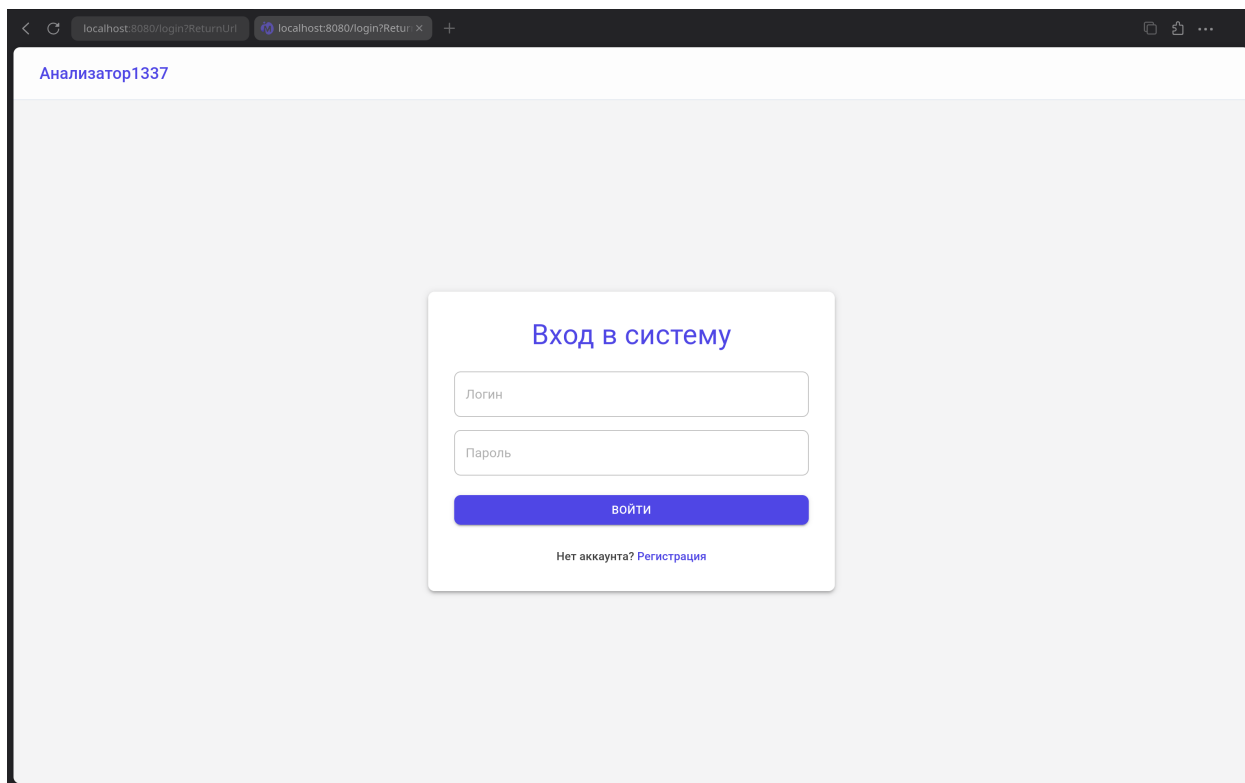


Рисунок 5 – Страница авторизации

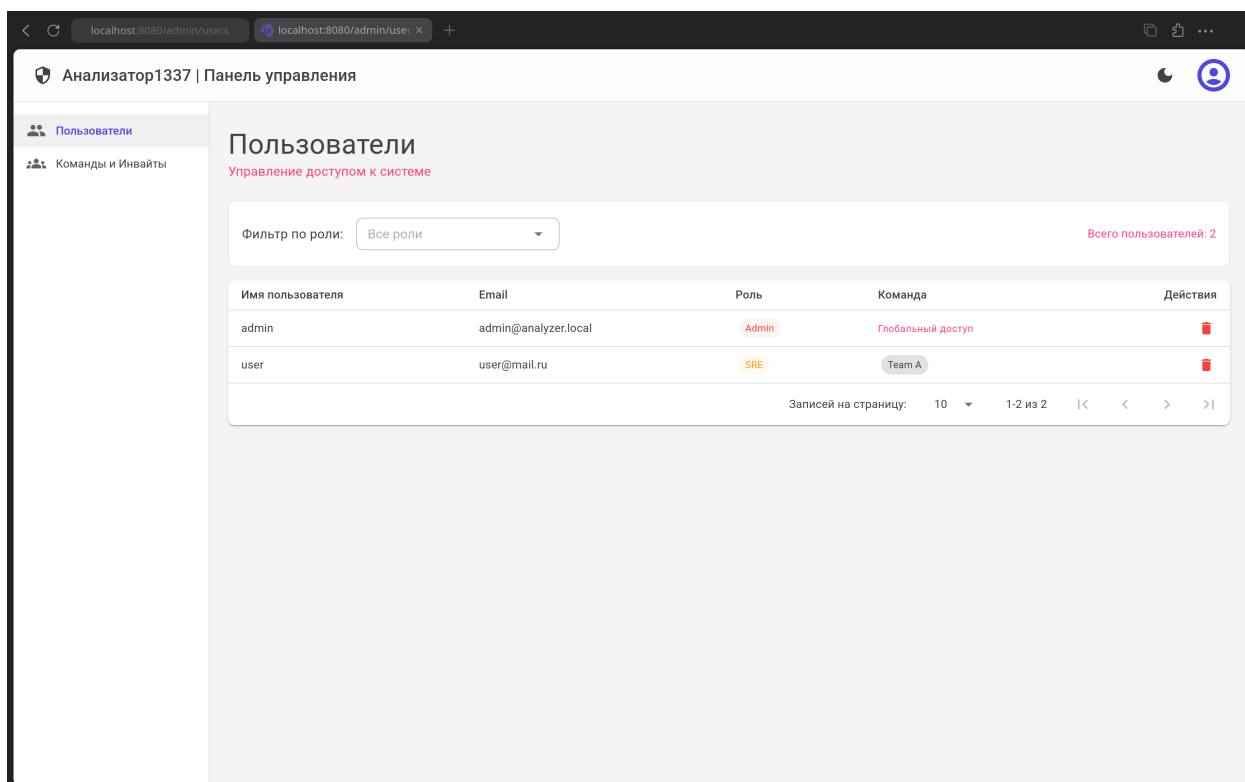


Рисунок 6 – Страница управления пользователями в панели администратора

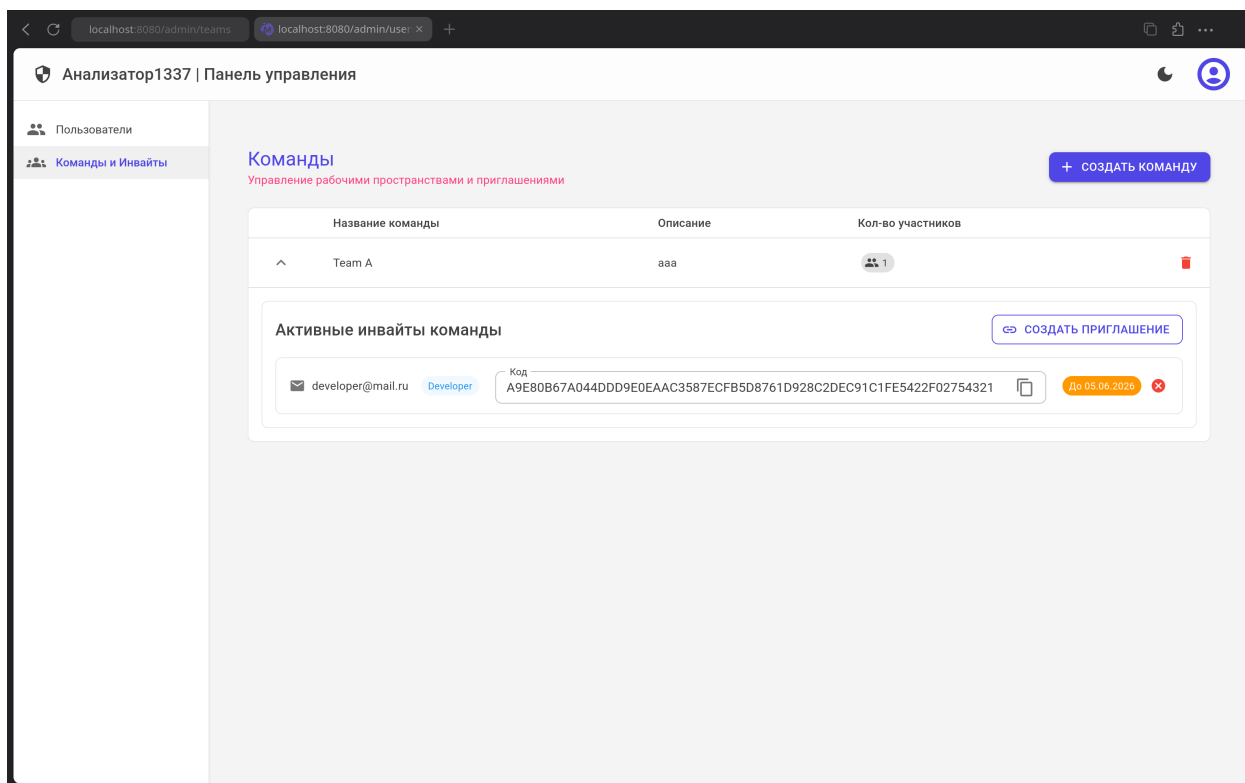


Рисунок 7 – Страница управления командами в панели администратора

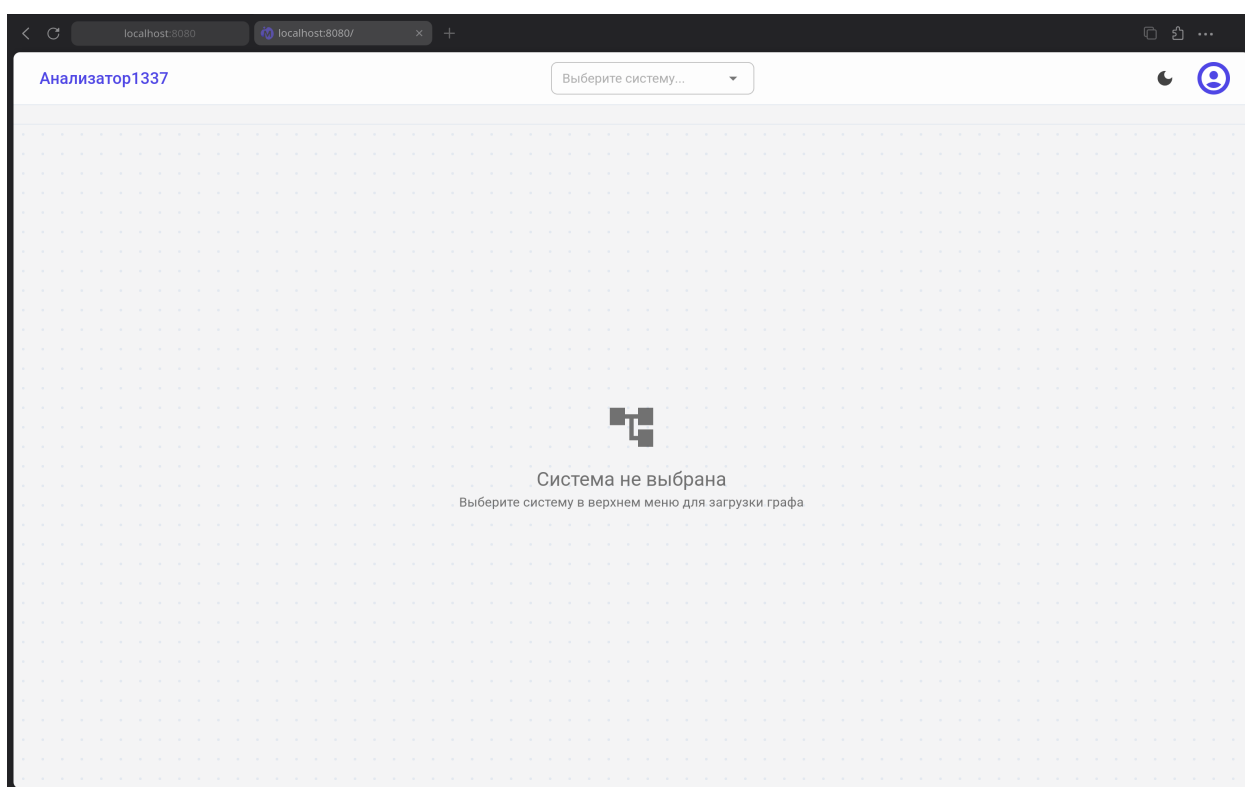


Рисунок 8 – Главная страница пользователя с ролью SRE

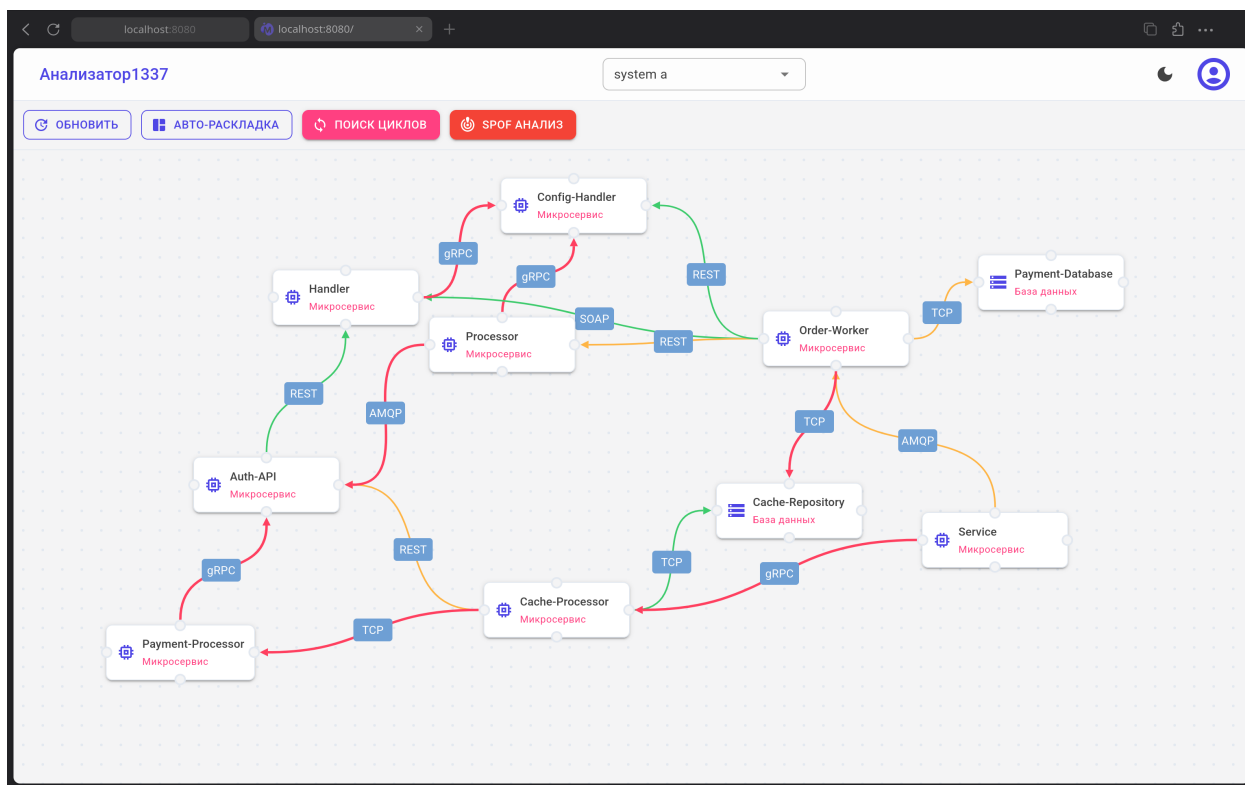


Рисунок 9 – Визуализация топологии системы

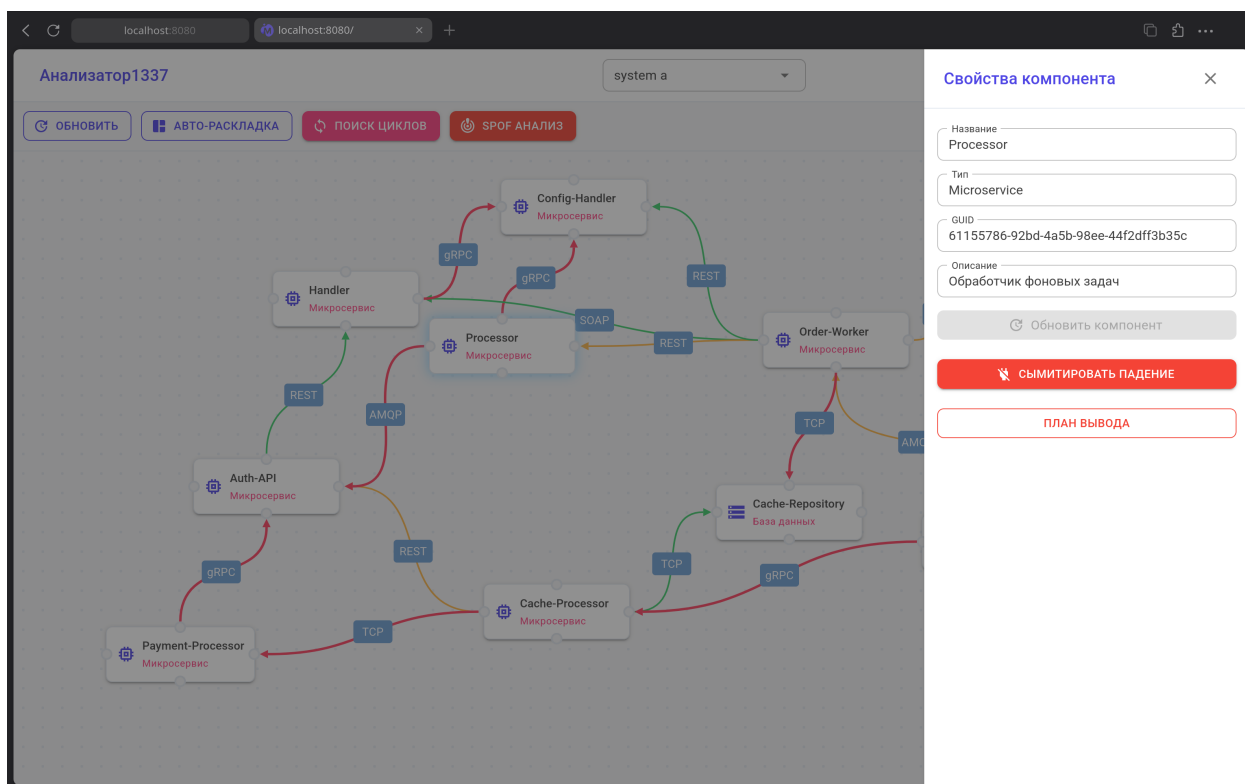


Рисунок 10 – Панель свойств компонента

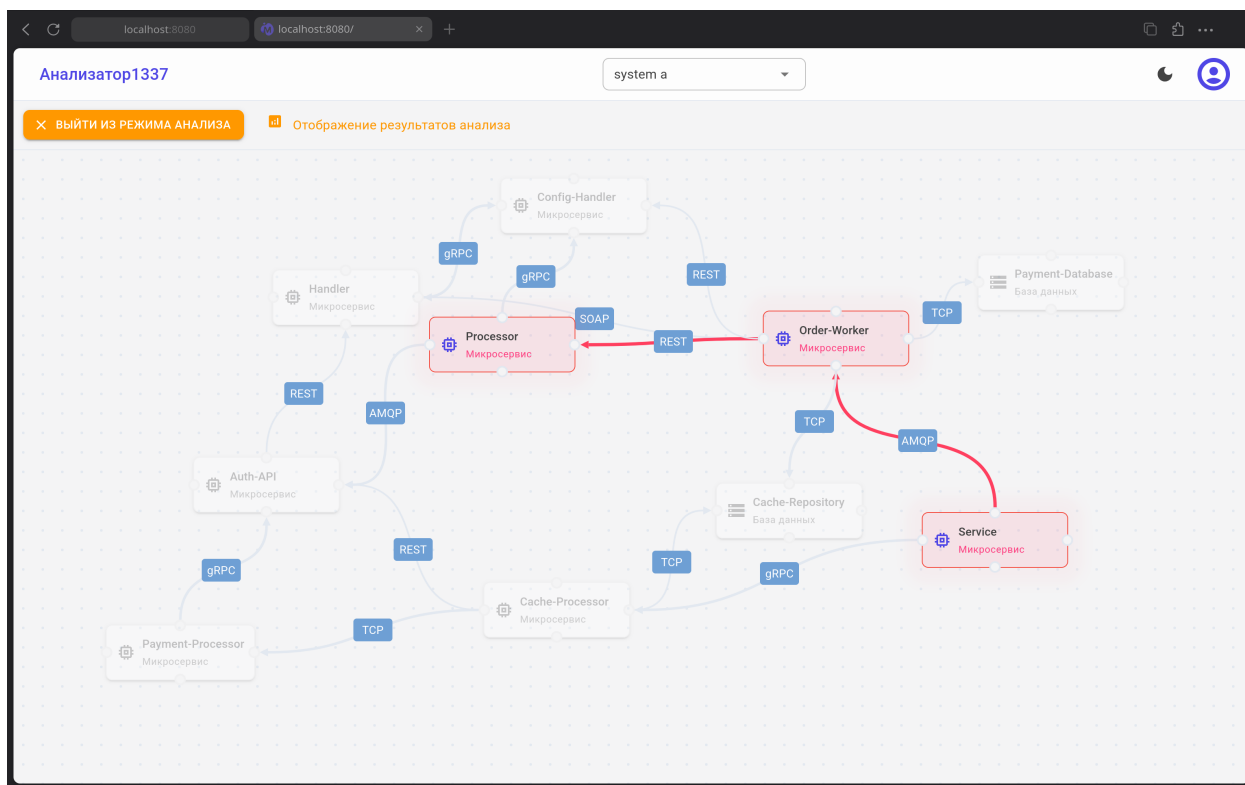


Рисунок 11 – Визуализация каскадного сбоя

The screenshot shows the 'Анализатор1337' application interface for the 'Мой профиль' (My Profile) page. The browser address bar shows 'localhost:8080/me'. The page title is 'Мой профиль'. The user information section includes:

- A profile icon with the letter 'U' and the name 'user'.
- A role indicator: 'Роль: SRE'.
- An ID: 'ID Команды: 59964fd2-45fc-4511-ad5d-3b6897d1f563'.
- Input fields for 'Имя пользователя' (Username) with the value 'user' and 'Email адрес' (Email address) with the value 'user@mail.ru'.
- A 'Сохранить изменения' (Save Changes) button.

 The security section, titled 'Безопасность' (Security), includes:

- A warning message: 'Регулярно меняйте пароль для обеспечения безопасности вашего аккаунта.' (Regularly change your password to ensure the security of your account).
- Input fields for 'Текущий пароль' (Current password), 'Новый пароль' (New password), and 'Подтвердите новый пароль' (Confirm new password).
- An 'Обновить пароль' (Update Password) button.

Рисунок 12 – Страница профиля пользователя

Выводы по технологическому разделу

В данном разделе был произведен выбор используемых СУБД и средств реализации приложения. Также был описан процесс создания объектов реляционной СУБД и были разработаны запросы в графовую СУБД для проведения анализа отказоустойчивости. Была реализована ролевая модель на уровне приложения и описана методика проведения тестирования. Также был продемонстрирован интерфейс разработанного приложения.

4 Исследовательский раздел

В данном разделе будут описаны проведенные исследование временной эффективности разработанной системы, исследуемыми параметрами являются время выполнения анализа каскадных сбоев в зависимости от количества узлов системы и время поиска циклов в системе.

4.1 Технические характеристики

Замеры времени проводились на ноутбуке со следующими характеристиками:

- ядро операционной системы — GNU/Linux 7.0.10_arch1-1 x86_64 [21];
- объём оперативной памяти — 32 ГБ;
- процессор — Intel® Core™ Ultra 7 155H с тактовой частотой 4.80 ГГц;
- количество физических ядер — 16;
- количество логических ядер — 22.

При замерах процессорного времени ноутбук был подключён к сети электропитания. Во время тестирования процессор был загружен только выполнением системных процессов, а также системы тестирования.

4.2 Технология замеров времени

Для каждого описанного далее исследования проводилось 100 замеров времени для каждого набора входных данных, полученные результаты усреднялись. Для получения времени выполнения запроса использовались временные метки между началом и концом выполнения запроса. Временные метки получены из СУБД Neo4j, результирующим временем считается разница времени между метками.

4.3 Первое исследование

4.3.1 Описание исследования

Данное исследование направлено на выявление зависимости времени выполнения поиска каскадных зависимостей от количества узлов в системе.

Условия исследования:

- количество циклов в графе системы зафиксировано;
- количество узлов в графе системы изменяется от 1000 до 10000 с шагом 100;
- количество связей в графе системы равняется количеству узлов, умноженному на два, на каждом шаге.

Предположение: с увеличением количества узлов, время выполнения анализа будет расти.

4.3.2 Результаты исследования

В таблице 14 представлена выжимка данных, полученных в результате проведения первого исследования. Полные данные, полученные в результате проведения исследования, представлены в таблице Б.1. На рисунке 13 представлен график зависимости времени выполнения анализа от количества узлов системы.

Таблица 14 – Результаты проведения первого исследования

Количество узлов	Время выполнения анализа, мс
1000	3.5
2000	3.7
3000	6
4000	7.5
5000	16.4
6000	7.6
7000	11.9
8000	15.4
9000	14.5
10000	15.5

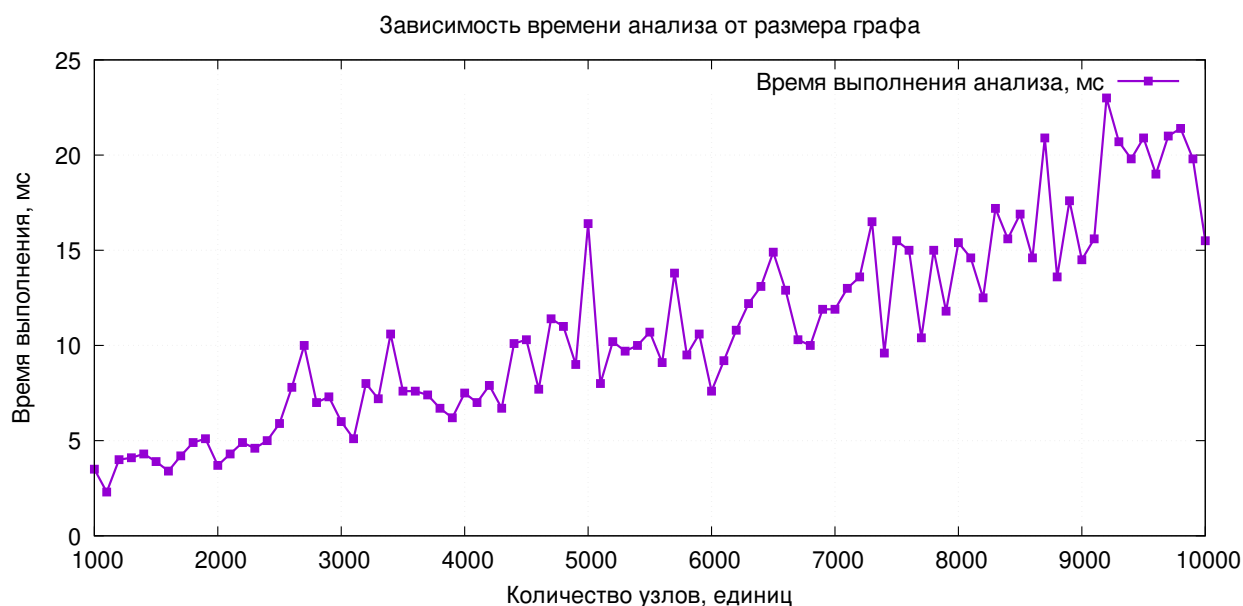


Рисунок 13 – Зависимость времени выполнения анализа от количества узлов системы

4.3.3 Вывод

Несмотря на зашумленность графика, прослеживается зависимость времени выполнения анализа от количества узлов системы: время выполнения незначительно увеличивается при росте количества узлов: при десятикратном увеличении количества узлов, время выполнения анализа выросло в 3 раза. Абсолютные значения времени достаточно малы — единицы и десятки миллисекунд, что позволяет использовать разработанную систему и БД на графах с тысячами вершин.

4.4 Второе исследование

4.4.1 Описание исследования

Данное исследование направлено на выявление зависимости времени выполнения поиска циклических зависимостей от количества циклов в системе. Условия исследования:

- количество циклов в графе изменяется от 10 до 50 с шагом 1;
- количество узлов рассчитывается как $50 + C_{cnt} \cdot 3$, где C_{cnt} — количество циклов в графе;
- количество связей в графе системы равняется количеству узлов,

умноженному на два, на каждом шаге.

Предположение: с увеличением количества циклов, время выполнения анализа будет расти.

4.4.2 Результаты исследования

В таблице 15 представлена выжимка данных, полученных в результате проведения второго исследования. Полные данные, полученные в результате проведения исследования, представлены в таблице Б.2. На рисунке 14 представлен график зависимости времени подсчета циклов от количества циклов в графе.

Таблица 15 – Результаты проведения второго исследования

Количество циклов	Время подсчета циклов, мс
10	8.4
15	10.7
20	13.0
25	13.9
30	22.2
35	26.2
40	32.2
45	38.5
50	41.5

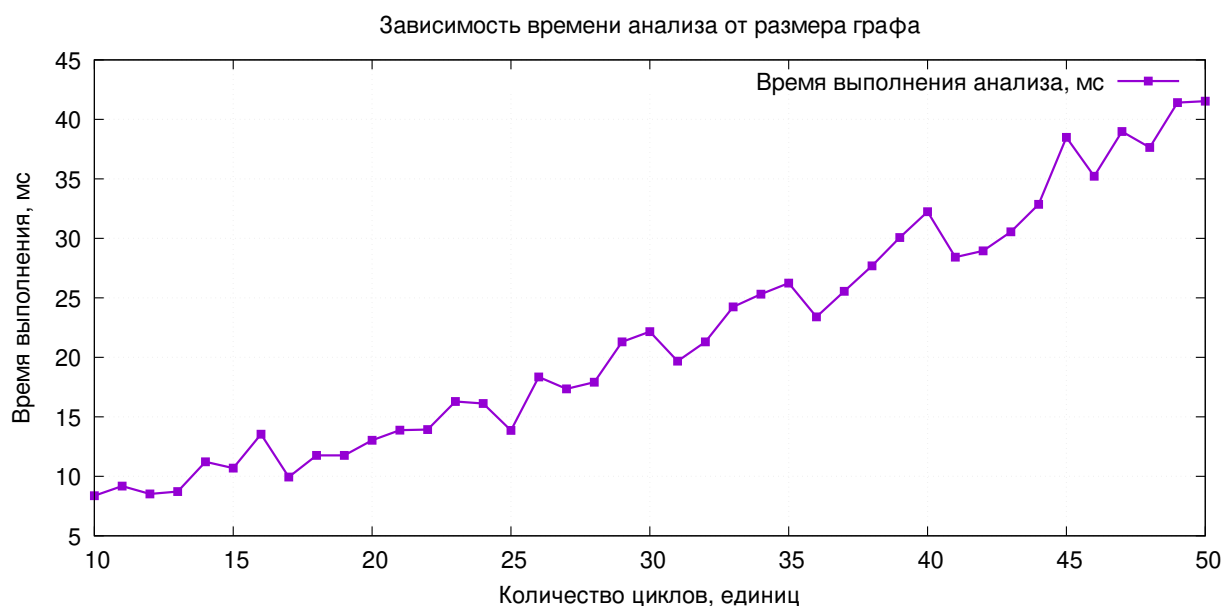


Рисунок 14 – Зависимость времени подсчета циклов от количества циклов в графе

4.4.3 Вывод

На представленном графике прослеживается зависимость времени поиска циклов в графе от количества циклов в системе: время выполнения увеличивается при росте количества циклов: при десятикратном увеличении количества узлов, время выполнения анализа выросло в 5 раз. Абсолютные значения времени достаточно малы — и десятки миллисекунд, что позволяет использовать разработанную систему и БД на графах с сотнями вершин и десятками циклов.

Выводы по исследовательскому разделу

В данном разделе была проведена оценка производительности выбранной графовой базы данных при работе с топологиями различной сложности. Операции глубокого обхода дерева для выявления каскадных зависимостей и поиск всех замкнутых контуров в ориентированном графе относятся к классу вычислительно сложных задач. В традиционных системах хранения такие алгоритмы требуют значительных вычислительных мощностей и времени на выполнение рекурсивных проверок.

Результаты проведенных исследований продемонстрировали высокую

временную эффективность используемой графовой системы управления базами данных. При масштабировании инфраструктуры до десяти тысяч узлов, а также при значительном увеличении количества циклических связей в системе, время обработки аналитических запросов возрастает предсказуемо и остается в пределах нескольких десятков миллисекунд.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы поставленная цель достигнута — была успешно спроектирована и разработана база данных для анализа отказоустойчивости систем на основе микросервисной архитектуры, а также реализовано веб-приложение, предоставляющее пользовательский интерфейс для работы с ней.

Все поставленные задачи выполнены:

- проведен детальный анализ предметной области обеспечения надежности распределенных систем, а также существующих систем мониторинга, подтвердивший актуальность разработки инструмента для статического анализа архитектурных уязвимостей;
- сформулированы требования к системе и обоснован выбор гибридного подхода к хранению информации: использование реляционной СУБД PostgreSQL для управления доступом и метаданных и графовой СУБД Neo4j для оптимизированной работы с топологией;
- разработана ролевая модель управления правами доступа пользователей на уровне приложения, обеспечивающая безопасную командную работу и изоляцию проектов;
- спроектированы структуры реляционной базы данных и модели графа свойств, а корректность реляционной схемы подтверждена успешным прохождением проверки на соответствие требованиям третьей нормальной формы;
- выбраны средства разработки: язык программирования C#, платформу .NET 10, фреймворк Entity Framework Core и расширение процедур АРОС для Neo4j;
- описана методика и проведено автоматизированное тестирование программного обеспечения путем написания модульных тестов с использованием библиотеки Moq и интеграционных тестов баз данных в xUnit;
- проведено исследование временной эффективности разработанных алгоритмов, которое подтвердило высокую производительность и масштабируемость графовой базы данных на топологиях, содержащих до десяти тысяч узлов и десятки циклических зависимостей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Микросервисная архитектура [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/pro-mikroservisnaya-arhitektura-cto-eto/> (дата обращения: 10.03.2026).
2. ITRS Geneos [Электронный ресурс]. – URL: <https://www.itrsgroup.com/platform/geneos> (дата обращения: 14.03.2026).
3. Prometheus — Monitoring system & time series database [Электронный ресурс]. – URL: <https://prometheus.io> (дата обращения: 14.03.2026).
4. Grafana: The open and composable observability platform [Электронный ресурс]. – URL: <https://grafana.com> (дата обращения: 14.03.2026).
5. Модели баз данных [Электронный ресурс]. – URL: <https://library.kuzstu.ru/dl.php?n=239752.pdf&type=nstu:common> (дата обращения: 16.03.2026).
6. PostgreSQL [Электронный ресурс]. – URL: <https://www.postgresql.org/> (дата обращения: 07.04.2026).
7. MySQL [Электронный ресурс]. – URL: <https://www.mysql.com/> (дата обращения: 07.04.2026).
8. Microsoft SQL Server [Электронный ресурс]. – URL: <https://www.microsoft.com/ru-ru/sql-server> (дата обращения: 07.04.2026).
9. Oracle Database [Электронный ресурс]. – URL: <https://www.oracle.com/database/> (дата обращения: 07.04.2026).
10. Neo4j Graph Intelligence Platform [Электронный ресурс]. – URL: <https://neo4j.com/> (дата обращения: 09.04.2026).

11. arangodb/arangodb [Электронный ресурс]. – URL: <https://github.com/arangodb/arangodb> (дата обращения: 09.04.2026).
12. OrientDB [Электронный ресурс]. – URL: <https://orientdb.dev/> (дата обращения: 09.04.2026).
13. C# [Электронный ресурс]. – URL: <https://dotnet.microsoft.com/ru-ru/languages/csharp> (дата обращения: 12.04.2026).
14. .NET 10 [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/core/whats-new/dotnet-10/overview> (дата обращения: 12.04.2026).
15. Entity Framework Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/ef/core/> (дата обращения: 12.04.2026).
16. Интегрированный язык запросов (LINQ) [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/linq/> (дата обращения: 12.04.2026).
17. Neo4j .NET Driver [Электронный ресурс]. – URL: <https://github.com/neo4j/neo4j-dotnet-driver> (дата обращения: 12.04.2026).
18. APOC Core 2026.04 Documentation [Электронный ресурс]. – URL: <https://neo4j.com/docs/apoc/current/> (дата обращения: 18.04.2026).
19. Introduction - Cypher Manual [Электронный ресурс]. – URL: <https://neo4j.com/docs/cypher-manual/current/introduction/> (дата обращения: 12.04.2026).
20. xUnit.net: Home [Электронный ресурс]. – URL: <https://xunit.net/> (дата обращения: 18.04.2026).

21. Arch Linux [Электронный ресурс]. – URL: <https://archlinux.org/>
(дата обращения: 19.05.2026).

ПРИЛОЖЕНИЕ А

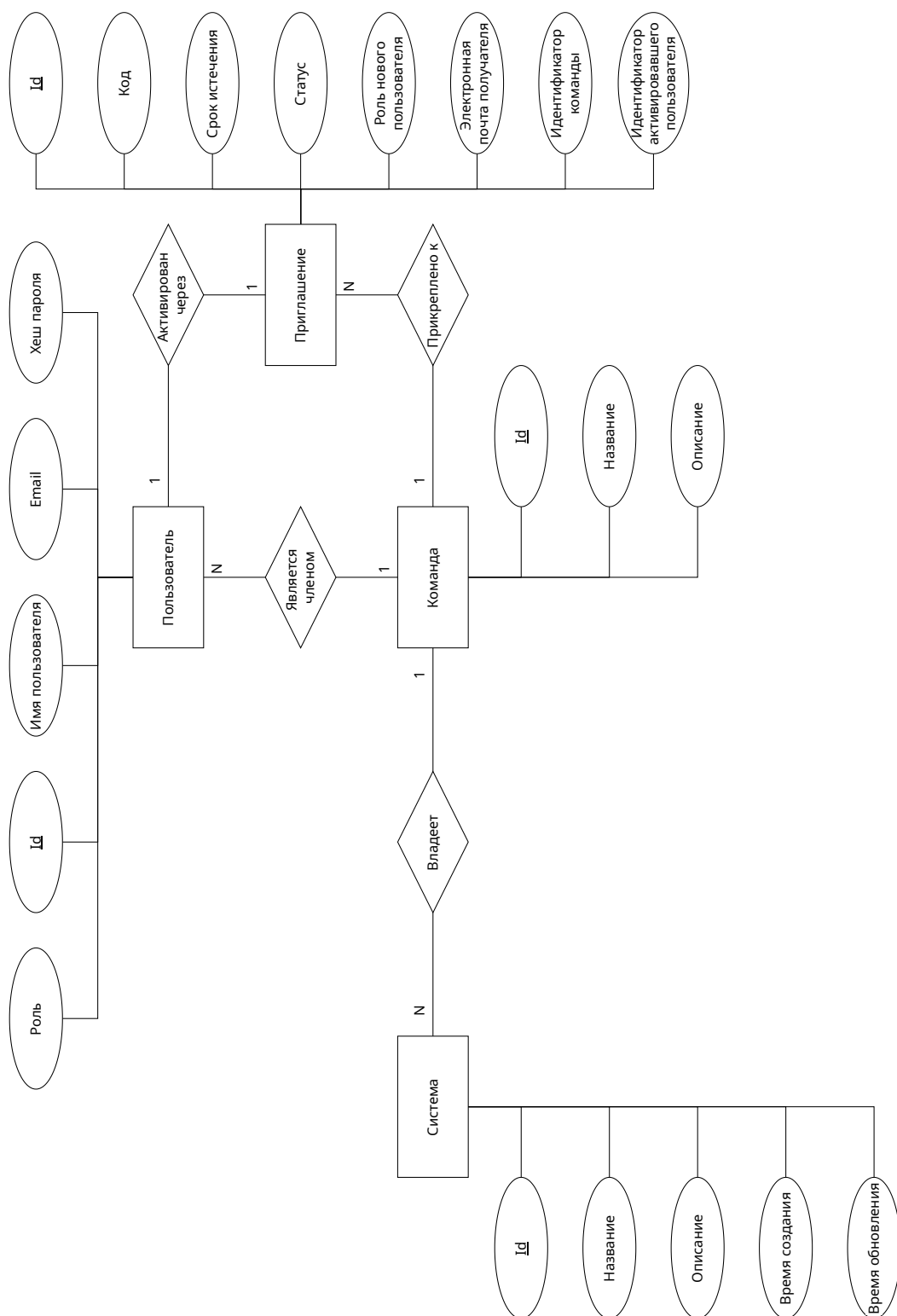


Рисунок А.1 – Диаграмма сущность-связь ролевой модели

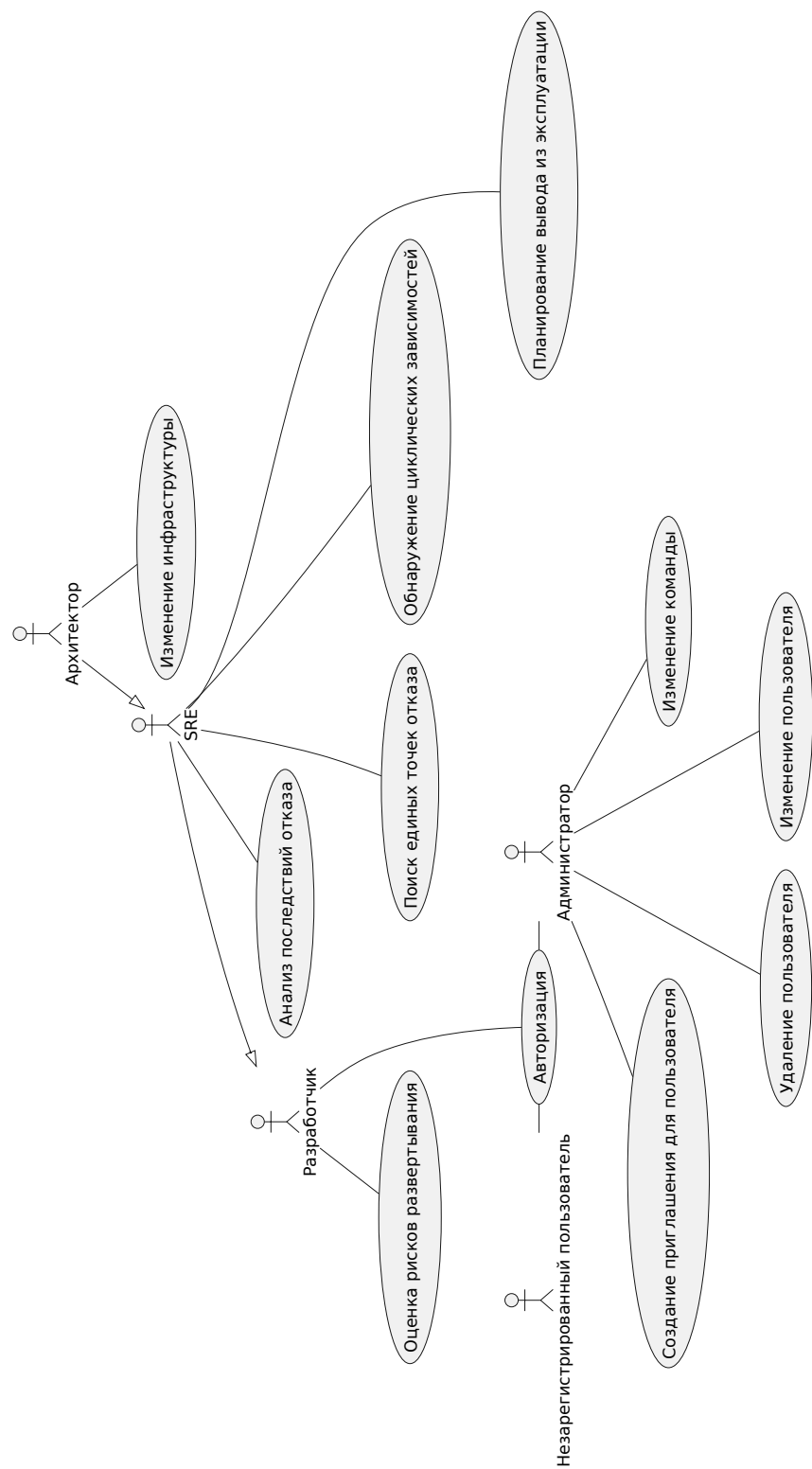


Рисунок А.2 – Диаграмма прецедентов

ПРИЛОЖЕНИЕ Б

Таблица Б.1 – Результаты проведения первого исследования

Количество узлов	Время выполнения анализа, мс
1000	3.5
1100	2.3
1200	4
1300	4.1
1400	4.3
1500	3.9
1600	3.4
1700	4.2
1800	4.9
1900	5.1
2000	3.7
2100	4.3
2200	4.9
2300	4.6
2400	5
2500	5.9
2600	7.8
2700	10
2800	7
2900	7.3
3000	6
3100	5.1
3200	8
3300	7.2
3400	10.6
3500	7.6
3600	7.6
3700	7.4

Продолжение таблицы Б.1

Количество узлов	Время выполнения анализа, мс
3800	6.7
3900	6.2
4000	7.5
4100	7
4200	7.9
4300	6.7
4400	10.1
4500	10.3
4600	7.7
4700	11.4
4800	11
4900	9
5000	16.4
5100	8
5200	10.2
5300	9.7
5400	10
5500	10.7
5600	9.1
5700	13.8
5800	9.5
5900	10.6
6000	7.6
6100	9.2
6200	10.8
6300	12.2
6400	13.1
6500	14.9
6600	12.9
6700	10.3
6800	10

Продолжение таблицы Б.1

Количество узлов	Время выполнения анализа, мс
6900	11.9
7000	11.9
7100	13
7200	13.6
7300	16.5
7400	9.6
7500	15.5
7600	15
7700	10.4
7800	15
7900	11.8
8000	15.4
8100	14.6
8200	12.5
8300	17.2
8400	15.6
8500	16.9
8600	14.6
8700	20.9
8800	13.6
8900	17.6
9000	14.5
9100	15.6
9200	23
9300	20.7
9400	19.8
9500	20.9
9600	19
9700	21
9800	21.4
9900	19.8

Продолжение таблицы Б.1

Количество узлов	Время выполнения анализа, мс
10000	15.5

Таблица Б.2 – Результаты проведения второго исследования

Количество циклов	Время подсчета циклов, мс
10	8.4
11	9.2
12	8.5
13	8.7
14	11.2
15	10.7
16	13.5
17	9.9
18	11.8
19	11.8
20	13.0
21	13.9
22	13.9
23	16.3
24	16.1
25	13.9
26	18.4
27	17.4
28	17.9
29	21.3
30	22.2
31	19.7
32	21.3
33	24.2
34	25.3

Продолжение таблицы Б.2

Количество циклов	Время подсчета циклов, мс
35	26.2
36	23.4
37	25.6
38	27.7
39	30.0
40	32.2
41	28.4
42	28.9
43	30.6
44	32.9
45	38.5
46	35.2
47	38.9
48	37.6
49	41.4
50	41.5

ПРИЛОЖЕНИЕ В

Презентация для курсовой работы на тему «Разработка базы данных для анализа отказоустойчивости систем на основе микросервисной архитектуры» состоит из 14 слайдов.